

A Large Neighbourhood Search Algorithm for Solving Container Loading Problems

Özge Şafak, Güneş Erdoğan *

School of Management, University of Bath, BA1 7AY, Bath, UK

ARTICLE INFO

Keywords:

Container loading problem
Large neighbourhood search

ABSTRACT

This paper presents a Large Neighbourhood Search (LNS) algorithm that finds an effective packing of a set of items into containers. The aim of the LNS algorithm is to provide packing solutions that can be adopted by many logistics companies. The LNS algorithm can handle a set of side constraints such as orientation, stacking, and separation as well as weight limit, loading priorities, and stability to address the issues arising for loading the items. The packing solution is also visualised and animated using an Excel workbook to help the workers for loading of the items. Finally, the authors present a comparison between the performance of the LNS algorithm and the state-of-the-art algorithms from the literature based on benchmark data sets. The results demonstrate the quality of the solutions achieved by the LNS algorithm.

1. Introduction

Efficient loading of items into containers is central to many activities in logistical planning. This problem arises in many industrial applications such as loading of container ships and pallets, air cargo management, and warehouse management. Reducing the number of containers required and increasing the value of items loaded can result in significant profits, which is crucial for the survival of a company in a competitive market. According to Bowes (2021) Parcel Shipping Index, global shipping volume exceeded 100 billion parcels in 2020 and is expected to more than double by 2026. Covid-19 crisis also had a booster effect on the parcel shipment volume due to the significant increase in the number of e-commerce shoppers. Based on these trends, optimising the packing of items into containers is evermore crucial to transport more items while simultaneously reducing the costs.

The problem considered in this paper is a multi Container Loading Problem (CLP) which aims to pack a set of given items into containers so that the total value of the items packed is maximised (Junqueira et al., 2012a). The basic variant of the CLP only contains constraints about non-overlap of packed items and the boundaries of the containers. In many industrial applications, CLP needs to be enriched with side constraints e.g., load stability, item-container compatibility, and orientation constraints. However, these constraints are likely to increase the complexity of the problem at hand, since the basic CLP is known to be NP-Hard (Pisinger, 2002). Consequently, finding an optimal packing of items in the containers while satisfying a variety of real world constraints is a non-trivial task.

In this paper, we present a Large Neighbourhood Search (LNS) algorithm that provides near optimal solutions to multi CLPs in reasonable computing time, which is crucial for practical applications. We have implemented the LNS algorithm into an Excel workbook named CLP Spreadsheet Solver, which stores item and container data, efficiently determines the packing of items into the containers, and graphically presents the results. CLP Spreadsheet Solver is open-source; it may be used, modified, and distributed freely, and is available upon request from the second author.

The main contributions of this paper are:

- To the best of our knowledge, this is the first study which presents a Large Neighbourhood Search (LNS) algorithm that can efficiently solve variants of multi container loading problems depending on the degree of heterogeneity of the items and containers.
- Our LNS algorithm can effectively handle a set of side constraints such as orientation, stacking, and separation as well as weight limit, loading priorities, and stability.
- We present an Excel spreadsheet that efficiently determines the packing of items into the containers and visualising and animating the packing solution in 3D.

The rest of the paper is organised as follows. In Section 2, we review the related literature on the CLP and its variants. In Section 3, we define the CLP along with the side constraints that we accommodate. We present our LNS algorithm in Section 4. Section 5 provides

* Corresponding author.

E-mail addresses: osa33@bath.ac.uk (Ö. Şafak), G.Erdogan@bath.ac.uk (G. Erdoğan).

computational results showing the quality of our solutions on benchmark instances, comparing them with the results from the literature. Section 6 concludes the paper along with future research directions.

2. Literature review

In this section, we provide an analysis of the literature on the CLP based on five features. First, we address the set of side constraints that arise in practice. Next, we review some constructive heuristics that provide initial solutions as well as improvement heuristics along with metaheuristics for the CLP. We also review some exact approaches. Finally, we provide a brief review of the existing studies for the solution visualisation of CLP.

2.1. Constraints in container loading

As pointed out in the survey of Zhao et al. (2016) who review 113 papers that cover all variants of the CLP, there is a lack of studies which incorporate a set of real-world constraints. For a comprehensive review of such constraints, we refer the reader to Bortfeldt and Wäscher (2013), wherein the authors categorise them related to the container, items, cargo, positioning, and load.

Weight limit constraint, which is crucial whenever heavy items have to be loaded, requires that the sum of the weights of the loaded items must be less than or equal to the given weight limit of the container. Terno et al. (2000), Bortfeldt and Gehring (2001), Egeblad et al. (2010), Fuellerer et al. (2010), and Correcher et al. (2017) are examples of studies in which the weight constraint for the stowed load is incorporated into the CLP.

Weight distribution constraints ensure that weight is distributed along the container. This is significantly important for handling and shipping the loaded container, since an unbalanced load may cause item shifts, thereby resulting in damaged items (see Bortfeldt and Wäscher (2013)). Eley (2002) aims to achieve an even weight distribution along the container length since this side defines the longest container edge. More recently, Ramos et al. (2018) adopt vehicle specific diagrams that show the admissible cargo weight as a function of the position of the centre of gravity of the cargo, thus complying with the regulations. Júnior et al. (2019) desire to have the load's centre of gravity close to the geometric centre of the container.

When all items cannot be loaded due to insufficient capacity, loading priorities of items come into play. Some items containing products with a closer due date or expiration date may have greater priority than others (Junqueira et al., 2012a). According to Bortfeldt and Wäscher (2013), although loading priorities are very important, they have not been paid enough attention in the design of algorithms. As an example, Ren et al. (2011) set the priority of items to maximise volume utilisation.

There are six orientations for an item. As stated by Bortfeldt and Wäscher (2013), it is common practice that the vertical orientation is restricted to prevent the items from being damaged or to ensure the stability of the load. Chien and Deng (2004), Gendreau et al. (2006), and Fuellerer et al. (2010) allow only one vertical orientation with 90° rotations of the items on the horizontal plane.

Stacking constraints (also known as load bearing constraints) restrict how items are placed on top of each other (see Bortfeldt and Wäscher (2013)). In the literature, stacking constraints are introduced in several ways. Terno et al. (2000) and Sciomachen and Tanfani (2007) prohibit an item of a particular type being placed on top of another type. Gendreau et al. (2006) and Fuellerer et al. (2010) require that no nonfragile item can be placed on top of a fragile one, although fragile items can be stacked on top of each other. On the other hand, Junqueira et al. (2012a) require that items cannot be placed above a fragile item. Junqueira et al. (2012a) limit the pressure of the items placed above an item of a particular type. Similarly, Júnior et al.

(2019) ensure that an item placed below other items must have enough resistance to support the pressure that is applied over its top face.

Complete shipment constraints require that if any part of a certain subset is loaded, then all other items belonging to it must be loaded (see Bischoff and Ratcliff, 1995; Eley, 2003; Bortfeldt and Wäscher, 2013).

Separation constraints prohibit certain items being loaded into the same container, e.g. foodstuff and perfumery articles should be transported separately (Eley, 2003). According to Bortfeldt and Wäscher (2013), separation constraints have not received much attention from researchers.

Positioning constraints restrict the position of items within the container (Bortfeldt and Wäscher, 2013). In this context, multi-drop constraints are commonly used in practice in such a way that items are loaded by taking into account the delivery route of the vehicle. As Zhao et al. (2016) state the aim of the multi-drop constraints is to avoid unnecessary unloading and reloading of large items at every destinations, see Terno et al. (2000), Gendreau et al. (2006), Fuellerer et al. (2010), Iori and Martello (2010), Junqueira et al. (2012b), Ceschia and Schaerf (2013) and Pollaris et al. (2016).

Load stability can be distinguished between vertical and horizontal stability. Vertical stability, which prevents items from falling down, is achieved through supporting the bottom of items by either the container floor or by the items underneath (Bortfeldt and Wäscher, 2013). While several papers (e.g. Eley, 2002; Liu et al., 2011; Zhang et al., 2012; Junqueira et al., 2012a; Ceschia and Schaerf, 2013; Liu et al., 2014) guarantee full support, others (e.g. Gendreau et al., 2006; Fuellerer et al., 2010) require partial support, at least as much the pre-specified minimum fraction of the base area. In addition, horizontal stability avoids shifting of items significantly during transport. Full horizontal stability ensures that each item is either horizontally adjacent to another item or a container wall (Bortfeldt and Wäscher, 2013). Liu et al. (2011) only requires that at least one side of the item is to touch the walls of the container or the sides of other items.

2.2. Constructive heuristics

There is a wide range of heuristics to decide how to put items into the container. In this paper, we only analyse the most commonly used heuristics, namely, wall-building and layer-building.

George and Robinson (1980) introduce a wall-building algorithm by constructing vertical walls that fill the width and height of the container. Although the wall-building approach tackles a set of simpler two-dimensional problems instead of a CLP, the main disadvantage of this approach is to lose space when items do not fully utilise the depth of the wall they belong to Egeblad et al. (2010). Some examples in which the wall-building heuristic is used are Bortfeldt and Gehring (2001) and Pisinger (2002).

Bischoff and Ratcliff (1995) utilise the layer-building approach that fills the container in horizontal layers from the bottom of the container upwards. Their motivation for this approach is to achieve a high degree of stability in contrast to wall-building approach where the height of adjacent walls varies significantly or where gaps exist between them. Terno et al. (2000), Lim et al. (2012), and Júnior et al. (2019) also use the layer-building approach within their heuristic algorithm.

The column-building heuristic aims to stack up as many items as possible. The three constructive heuristics differ in terms of the next point, where an item can be added, they choose. Wall-building, layer-building, and column-building heuristics choose the rightmost, farthest from the front-side, and the topmost points respectively.

2.3. Improvement heuristics

In the literature, improvement heuristics have been commonly developed to improve solutions provided by constructive heuristics. Examples of popular metaheuristics are Genetic Algorithms (e.g. Bortfeldt and Gehring, 2001; Wu et al., 2010), Tabu Search (e.g. Liu et al. (2011)) and Simulated Annealing (e.g. Mack et al. (2004)). In addition to metaheuristics, a Greedy Randomised Adaptive Search Procedure (e.g. Moura and Oliveira (2005)) and a Variable Neighbourhood Search (e.g. Parreño et al. (2010)) are rarely used and combined with the constructive heuristics to find better quality solutions. For a comprehensive review of the improvement heuristics for the CLP, we refer the reader to Zhao et al. (2016).

2.4. Exact approaches

In contrast to heuristic methods, there are a few exact approaches in the literature (e.g. Chen et al. (1995), Fasano (2004), Hifi et al. (2010), Junqueira et al. (2012a), Tsai et al. (2015), and Paquay et al. (2016)). Silva et al. (2019) compare the exact approaches for CLPs. Recently, do Nascimento et al. (2021) develop an exact approach that considers the resolution of integer linear programming and constraint programming models iteratively to solve the single container loading problem.

2.5. Solution visualisation

Simultaneously determining the packing patterns and visualising the solution have been rarely developed. Chien and Deng (2004) address the difficulty of loading items into the containers for the workers. They underline that changing the packing patterns might be needed to load more items, and consequently unloading and reloading operations may take several days. To address this issue, they determine the container packing patterns (i.e., the packing order, the orientation and location of each item) and visualise the results. Chan et al. (2006) develop another output of loading plan for air-cargo pallets. Dereli and Daş (2010) develop a computational algorithm based on a swarm intelligence algorithm, a graphical user interface, and a simulation program visualising the actual packing process in 3D.

3. Problem definition

The CLP is formally defined as follows: Three-dimensional cuboid items are packed into three-dimensional rectangular containers subject to two constraints:

- Each item is stowed entirely within the container so that the edge of each item lies parallel to the edges of the container.
- No two items overlap.

Container loading is classified as the three-dimensional (3D) rectangular packing problem within the general cutting and packing problem (Chen et al., 1995). Classification of the cutting and packing problem is provided by Bortfeldt and Wäscher (2013) as: (1) there are enough containers are available to accommodate all items and (2) only a subset of the items can be packed due to insufficient capacity of the containers. Different objectives are pursued for problems of these two categories. In the first category the number of containers used is minimised, whereas in the second category container volume utilisation is maximised. According to the topology proposed by Wäscher et al. (2007), each category of the problem can be further distinguished with respect to the assortment of items and the assortment of the containers. Items can be identical in terms of their shape and dimensions. The assortment of items is considered to be weakly heterogeneous if it consists of relatively few different item types, and each item type contains large number of items. It is called strongly heterogeneous if there are large



Fig. 1. Default placement on the x/z surface.

number of different types, and each item type includes a few items. Containers can be identical, weakly, or strongly heterogeneous.

According to Wäscher et al. (2007) topology, for input minimisation type of problems, multiple stock-size cutting stock problem (MSSCSP) and multiple bin-size bin packing problem (MBSBPP) depending on the degree of heterogeneity of the items and containers can be effectively solved by the LNS algorithm. These problems were addressed in only 10% of the papers reviewed in Bortfeldt and Wäscher (2013).

Li et al. (2014) also classify 3D packing problems into off-line and on-line environments. Characteristics of input items are known beforehand in off-line problems, while decisions have to be made immediately once items arrive without full knowledge about their characteristics in on-line problems (Ali et al., 2022). Solution approaches for both off-line and on-line problems are reviewed in Ali et al. (2022). In this paper, we apply our approach to solve only off-line problems.

For many industrial applications, solely optimising the objective function such as maximising the container volume utilisation and minimising the number of containers used yields a limited practical value. As do Nascimento et al. (2021) point out the importance of practical constraints, they should be considered in the CLP. Otherwise, customer satisfaction may decrease due to damaged items or incomplete shipments. Therefore, as pointed out by Zhao et al. (2016), to benefit the research community as well as make this work more likely to be adopted by practitioners, we include a set of side constraints in the design of our LNS algorithm.

(C1) *Weight (capacity) limit*: The sum of the weights of the loaded items must be less than or equal to the given weight limit of the container.

(C2) *Loading priorities*: The loading priorities of the items are taken into account. Two sets of items are considered; (i) the first set includes the items that “must be packed”, and (ii) the second set includes the items that “may be packed”. Items that “must be packed” have the highest priority to be placed within the container.

(C3) *Orientation constraints*: Fig. 1 demonstrates our naming convention for the dimensions of the items and the containers, with the default placement on the x/z surface. Examples of all orientations for an item with width(x) = 10 cm, height(y) = 20 cm, and length(z) = 40 cm are provided in Fig. 2. The first and the third letters of the orientation show the surface the item is placed on, hence the default orientation is xyz . Any other order refers to a rotation where the associated letters imply the type of rotation, e.g. zyx refers to the case where the item is still resting on the x/z surface, with the item is rotated so that the width became the length.

In this work, it is assumed that item can be placed on the x/z surfaces (see Fig. 2(a) and (b)). The user defines whether an item is allowed to be placed on x/y surface (see Fig. 2(c) and (d)) and y/z surface (see 2 (e) and (f)). A 90° turn around its vertical axis is always possible.

(C4) *Stacking constraints*: Heavy items are only placed on the floor, since they may crush the items they are placed on. Moreover, no items are allowed to be placed on top of a *fragile* item, since which may be damaged by the weight of the item they support. An item can be both *heavy* and *fragile* at the same time.

(C5) *Separation constraints*: If an item is not *compatible* with another item, then they cannot be packed into the same container. Furthermore, the LNS algorithm can also handle the case of an item type not being compatible with a container type. For example, dairy items should not be loaded into a non-refrigerated container.

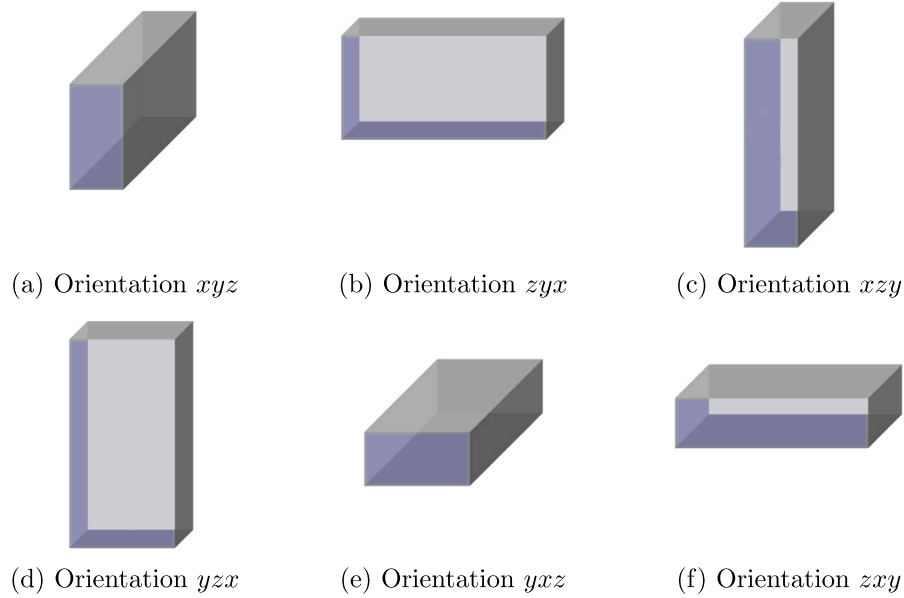


Fig. 2. Orientations for an item.

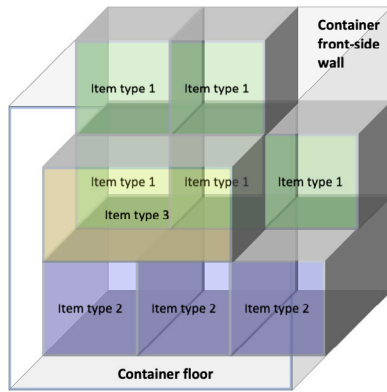


Fig. 3. Vertical & front-side supports.

(C6) *Stability*: Load stability is achieved through fully supporting the bottom of items by either the container floor or by other items underneath, called *vertical support*. In addition, the user may activate *front-side support*, requiring every item to be fully supported on its surface facing the front of the container (container's front-side wall) (see example in Fig. 3). The far walls (front-sides) of items of type 1 and type 2 are already supported by the container's front-side wall. The intersection area of item of type 3 and items of type 1 is equal to the area of the front-side of item of type 3.

In addition, the weight distribution is taken into account. If users choose to balance the load, the moment of the item weights with respect to the mid-line along z -axis (length) is minimised in the objective function.

In this paper, we consider a multi-objective CLP where the multi objectives are achieved by a lexicographical approach. Our primary objective is to maximise the profit of items packed, calculated as the total value of the items packed minus the cost of the containers used.

Table 1

Data for Fig. 4.

	Width (cm)	Height (cm)	Length (cm)	# of items
Item type 1	10	10	10	10
Item type 2	20	20	20	10
Item type 3	30	30	30	10
Item type 4	40	40	40	10

The secondary objective is to maximise total volume packed in the containers. The tertiary objective of our problem is minimising the sum of the moments of the items in the containers along the mid-line parallel to the z -axis, if balancing the items in the containers is desired by the user. Finally, the quaternary objective is to minimise the distance between the items of same type.

Our LNS algorithm packs items into the container using one of the wall-building, layer-building, and column-building heuristics which are the options of the constructive heuristic. Users can select one of these options and analyse their performances depending on the input data they have. Fig. 4 shows the arrangements of 40 items in a container (with dimensions width (x) = 120 cm, height (y) = 120 cm, length (z) = 120 cm) achieved by the LNS algorithm based on the wall-building, layer-building, and column-building heuristics, respectively. Table 1 also provides the dimensions of items as well as their required quantities. In this numerical example, items can be placed in all orientations. There is no weight limit, no loading priorities for the items, and there are no heavy or fragile items. We assume that all item types are compatible with each other and each item type is compatible with each container type. We ensure full vertical support, but assume that front-side support is not required.

While wall-building heuristic in Fig. 4(a) packs items by constructing vertical walls from the front of the container to the back, layer-building heuristic in Fig. 4(b) constructs horizontal layers from the bottom of the container to the top. In Fig. 4(c), column-building heuristic builds a column with as many items as possible.

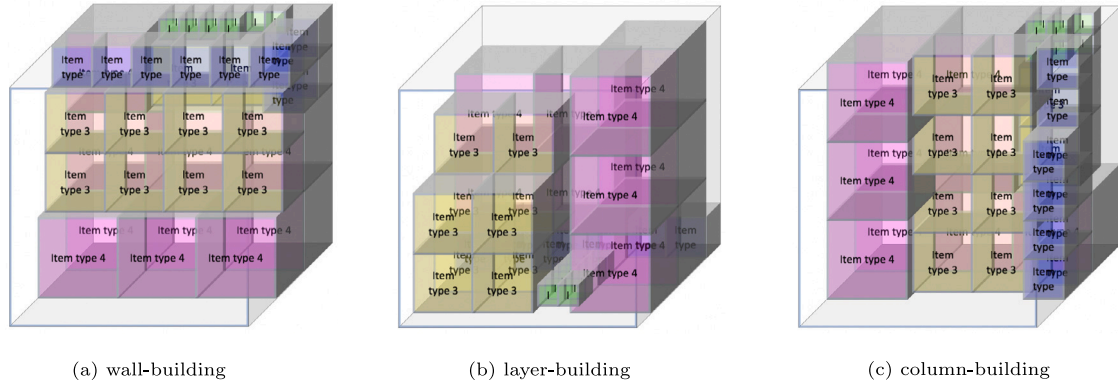


Fig. 4. Solutions with different constructive heuristics.

4. Solution algorithm

We develop an LNS algorithm to solve the multi CLPs. The algorithm gets item and container specific information as well as the compatibility data from the CLP Spreadsheet Solver. The compatibility data includes both whether item types are compatible with each other and whether an item type is compatible with a container type. The Large Neighbourhood Search Algorithm first checks the feasibility of the instance. The infeasibility is reported to the user under three conditions; (1) if the volume (weight) capacity of the containers are not enough to pack all mandatory items, (2) if any of the items is too wide, too tall or too long to fit into any container, (3) if at least one of the items is not compatible with any container. Then, the Large Neighbourhood Search Algorithm sorts the items with respect to their priority, size, and profit. Next, Constructive Heuristic (Algorithm 2) finds an initial solution based on one of the wall-building, layer-building, and column-building heuristics selected by the user. This is followed by the Improvement Heuristic (Algorithm 3) to find different solutions in the neighbourhood of the current solution through destructive and constructive algorithms. To that end, Improvement Heuristic randomly empties a number of containers and a subset of items from the rest of the containers. Next, Constructive Heuristic repacks removed items into the containers. Until the time limit is reached, Improvement Heuristic repeats the destruction, perturbation, and construction steps to find better solutions. Details of the steps are provided in the following subsections.

Algorithm 1: Large Neighbourhood Search Algorithm

Input: Item, container, and compatibility data
Step 1. Feasibility Check: Check the feasibility of the instance regarding the size and weight of items, container capacities, and compatibility data
Step 2. Sort Items: Sort the set of items I with respect to their priority, size, and profit.
Step 3. Constructive Heuristic: Add items to containers using the AddItemToContainer procedure based on the selection of wall-building, layer-building and column-building heuristics.
Step 4. Improvement Heuristic: While the time limit is not exceeded;
 Step 4.a. Destroy Solution: randomly empty a number of containers, and remove a subset of items from the rest of the containers,
 Step 4.b. Perturb Solution: randomly change the order of rotation for the items to be packed, and the order of containers to be used.
 Step 4.c. Reconstruct Solution: repack the removed items into the containers using Constructive Heuristic.
Output: Solution worksheet with item coordinates in the containers

4.1. Sorting the items

Items are sorted in decreasing order of their priority. Items that “must be packed” have the highest priority, which are again sorted with respect to the sort criterion provided by the user. If “volume” is selected, items are sorted by decreasing order of their volume. If items’ volumes are equal, they are sorted in decreasing order of their weight. If “weight” is selected, items are sorted in decreasing order of their weight and volume, respectively. If sort criterion is “max { width; height; length }”, items are sorted in decreasing order of their longest edge and volume, respectively. Finally, items that “may be packed” are sorted by decreasing order of the ratio of profit to volume.

4.2. Constructive heuristic

Algorithm 2 first invokes Sort Containers to determine the order of containers to fill. Containers are ordered in descending order of their priority, volume packed, and their cost. Next, for each container and for each item, Constructive Heuristic sorts the orientations in decreasing order of the area of the wall which can be constructed by those items of the same type. Let a container’s width (x) and height (y) be 5 cm and an item’s width, height and length be 2 cm, 2 cm, 3 cm, respectively. For the orientation xyz , the maximum area of the wall which can be constructed with items of that type will be $4 \times 4 = 16 \text{ cm}^2$. For the orientation zyx , the maximum area of the wall will be $4 \times 3 = 12 \text{ cm}^2$. Therefore, for this item, the orientation xyz will have a higher priority than the orientation zyx .

Algorithm 2 adds items to the containers using the procedure AddItemToContainer provided in Appendix D. Once items of a certain type are started to be loaded into a container, all items of the same type are attempted to be placed. If no more items of that type could be placed, then Algorithm 2 starts to add items of next type.

For each container $k \in C$, after all items are attempted to be packed into the container $k \in C$, if there is at least one item that “must be packed” but could not be placed, Algorithm 2 sets the feasibility of the incumbent solution to *FALSE*.

Next, dispersion and utilisation of the containers in the incumbent solution are calculated to better place the items in the improvement later. Dispersion is calculated as follows:

$$dispersion = \sum_{i \in I} c_i^2, \quad (1)$$

where c_i is the number of containers which include the item of type i , $i \in I$. For container $k \in C$, utilisation is calculated as follows:

$$utilisation = \sum_{k \in C} \max \left(\frac{container_volume_packed(k)}{container_volume_capacity(k)}, \frac{container_weight_packed(k)}{container_weight_capacity(k)} \right), \quad (2)$$

Algorithm 2: Constructive Heuristic

Input: Item, container, and compatibility data
 $IsItemAdded \leftarrow FALSE$
Sort Containers (Algorithm 5)
 $C \leftarrow$ set of containers
 $I \leftarrow$ set of item types

```

for  $k \in C$  do
  for  $i \in I$  do
    Sort the rotation order of the item of type  $i$  for the container
       $k \in C$ 
  end
  for  $i \in I$  do
     $IsItemAdded \leftarrow TRUE$ 
    while (There exists at least one item of type  $i \in I$  to be packed)
      and ( $IsItemAdded = TRUE$ ) do
       $IsItemAdded \leftarrow AddItemToContainer$ (item of type
         $i$ , container  $k$ ) (Algorithm 6)
      end
    end
     $incumbent\_feasible \leftarrow TRUE$ 
    for  $i \in I$  do
      if (There exists at least one item of type  $i \in I$  to be packed) and
        ( $mandatory(i) = TRUE$ ) then
         $incumbent\_feasible \leftarrow FALSE$ 
        exit for
      end
    end
    end
    Calculate dispersion for the incumbent solution
    Calculate utilisation of containers for the incumbent solution
    if incumbent solution is better than the best_known solution then
       $best\_known \leftarrow incumbent$ 
      for  $m \in C$  do
        if optimise balance = TRUE then
          Calculate moment ( $m$ ) using equation (4)
        end
        Calculate distance( $m$ ) using equation (5)
      end
       $best\_known\_moment := \sum_{m \in C} moment(m)$ 
       $best\_known\_distance := \sum_{m \in C} distance(m)$ 
    end
  end
end

```

Output: incumbent solution

where $container_volume_packed(k)$ and $container_weight_packed(k)$ are the total volume and weight of the items packed into the container $k \in C$ and $volume_capacity(k)$ and $weight_capacity(k)$ are the volume and weight limit of the container $k \in C$, respectively.

If one the following conditions holds:

1. $incumbent_feasible = TRUE$ and $best_known_feasible = FALSE$
2. $incumbent_feasible = FALSE$ and $best_known_feasible = FALSE$ and

$$incumbent_volume_packed > best_known_volume_packed$$
3. $incumbent_feasible = TRUE$ and $best_known_feasible = TRUE$ and

$$incumbent_profit > best_known_profit$$
4. $incumbent_feasible = TRUE$ and $best_known_feasible = TRUE$ and

$$incumbent_profit = best_known_profit \text{ and } incumbents_volume_packed > best_known_volume_packed$$

then Algorithm 2 updates the $best_known$ with the incumbent one. $incumbent_profit$ is the profit of items loaded in the incumbent solution. The multi-objective CLP is handled by a lexicographical approach applied through these conditions. Our primary objective is to maximise the profit. When both incumbent and $best_known$ solutions are feasible, the $best_known$ solution is updated only if the profit of the incumbent solution is bigger than the profit of the

$best_known$ solution. The secondary objective is to maximise the total volume of containers utilised. Therefore, when both incumbent and $best_known$ solutions are feasible and the profits of the incumbent and $best_known$ solutions are the same, the $best_known$ solution is updated only if the total volume packed in the incumbent solution is bigger than the total volume packed in the $best_known$ solution.

If users select the option of balancing items in the containers, Algorithm 2 calculates the moment for each container m with respect to the mid-point of the container floor (see Appendix C for the formula). In addition, Algorithm 2 calculates the distances between the items of same type in each container $m \in C$ as in (5) in Appendix C. Total distance for the incumbent solution is the sum of distances for all the containers. The LNS algorithm attempts to minimise the total distance, in other words, items are located in such a way that the items with the same type are close to each other as possible and at the same time outermost point of the items is minimised.

It is important to note that the solution of the Constructive Heuristic may not be feasible because all the items that are mandatory may not be packed. In this case, the solution is still passed to the Improvement Heuristic since an infeasible solution may turn into a feasible solution by reorganising the containers in the Improvement Heuristic.

4.2.1. Packing an item to a container

Packing starts with checking the feasibility of possible solution when an item with type $i \in I$ is added to the container $k \in C$:

1. if the item with type $i \in I$ is compatible with the type of the container $k \in C$.
2. if the volume capacity of the container $k \in C$ is still enough to pack the item of type $i \in I$.
3. if the weight capacity of the container $k \in C$ is still enough to pack the item of type $i \in I$.
4. if the item of type $i \in I$ is compatible with all the items already packed into the container $k \in C$.

If one of the conditions does not hold, item of type $i \in I$ is not added to the container $k \in C$. Only if all feasibility conditions are satisfied, we attempt to pack item of type $i \in I$ in its best orientation unless it is forbidden by the user. If it is forbidden, the next orientation in $O(i)$, where $O(i)$ is the orientation order set for the item of type i , is selected to pack the item.

We use the *addition point* concept to determine the possible positions in a container where the items can be placed. Let $P(k)$ be the set of addition points for the container $k \in C$ in the current solution. When the container k is empty, $P(k)$ only includes the point with coordinates (0, 0, 0), meaning that the first item is added to the farthest left lower corner. When an item is placed at an addition point, 3 new addition points are generated (closest left lower corner, farthest right lower corner, farthest left upper corner), at which the next items can be placed (see Fig. 5).

We set the origins ($origin_x$, $origin_y$, $origin_z$) of the item with type $i \in I$ to the coordinates of the addition point $p \in P(k)$. If $origin_y > 0$ for a heavy item, we look for the next addition point to place the heavy item, since we do not place a heavy item on another item. We also define the farthest coordinates ($opposite_x$, $opposite_y$ and $opposite_z$) of the item with type i as follows:

$$\begin{aligned}
 opposite_x &= \begin{cases} origin_x + width(i) & \text{for orientation } xyz, \\ origin_x + length(i) & \text{for orientation } zyx, \end{cases} \\
 opposite_y &= \begin{cases} origin_y + height(i) & \text{for orientation } xyz, \\ origin_y + height(i) & \text{for orientation } zyx, \end{cases} \\
 opposite_z &= \begin{cases} origin_z + length(i) & \text{for orientation } xyz, \\ origin_z + width(i) & \text{for orientation } zyx. \end{cases} \quad (3)
 \end{aligned}$$

Table 2
List of papers included in Tables 4–7.

Code	Paper	Code	Paper
I. (1989)	Ivancic et al. (1989)	B.M. (1990)	Bischoff and Marriott (1990)
N. (1994)	Ngoi et al. (1994)	B.R. (1995)	Bischoff and Ratcliff (1995)
G.B. (1997)	Gehring and Bortfeldt (1997)	B.G. (1998)	Bortfeldt and Gehring (1998)
B. (2000)	Bortfeldt (2000)	B.G. (2001)	Bortfeldt and Gehring (2001)
E. (2002)	Eley (2002)	E. (2003)	Eley (2003)
L. (2005)	Lim et al. (2005)	L.Z. (2005)	Lim and Zhang (2005)
M.O. (2005)	Moura and Oliveira (2005)	W. (2008)	Wang et al. (2008)
T. (2008)	Takahara (2008)	H.H. (2010)	He and Huang (2010)
C. (2011)	Che et al. (2011)	D.D. (2011)	Dereli and Das (2011)
R. (2011)	Ren et al. (2011)	L. (2012)	Lim et al. (2012)
Z. (2012)	Zhu et al. (2012)	C.S. (2013)	Ceschia and Schaerf (2013)
T.E.W.B. (2017)	Toffolo et al. (2017)	K.S.J.S.C. (2020)	Kurpel et al. (2020)

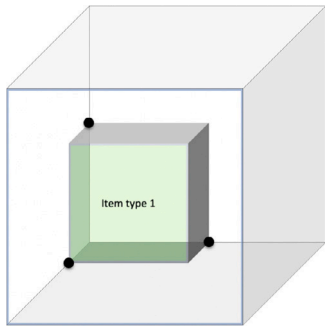


Fig. 5. Three new addition points on an item.

These coordinates can be computed similarly for other orientations.

For the addition point $p \in P$, we check whether the item of type $i \in I$ fits inside the container $k \in C$ through the conditions: $opposite_x \leq container_width(k)$, $opposite_y \leq container_height(k)$, and $opposite_z \leq container_length(k)$. If at least one of the corners stays outside of the container, we attempt to put the item of type $i \in I$ to the next addition point. Next, we check the feasibility regarding the corners of other items in the same container $k \in C$, i.e. they should not overlap with each other.

We guarantee vertical support and optional front-side support for the items. If $origin_y = 0$, the bottom of the item is already supported by the container's floor. Otherwise, the intersection area of the item of type $i \in I$ and items directly below it should be equal to the bottom area of the item of type $i \in I$. Similarly, if $origin_z = 0$, the front-side wall of the item is already supported by the container's front-side wall. Otherwise, the intersection area of the item of type i and the item which is in front of the item of type i should be equal to the area of the front-side of the item of type i , if front-side support option is selected by the user. If the conditions for vertical and optional front-side support cannot be satisfied, we attempt to place the item to the next addition point.

Algorithm 6 tries to fit item of type i to all addition points $p \in P(k)$ by taking into account the feasibility conditions. Then, the algorithm selects the best fit position among the candidate addition points according to the user's choice of constructive heuristic. If the wall-building is selected, the algorithm selects the deepest addition point. Among the addition points at deepest area, the algorithm selects the one at lower height. If there are more than one addition point with minimum depth and minimum height, the algorithm selects the one closer to the origin

of the container. If the user's selection is layer-building, the algorithm searches for the addition point with the minimum height, minimum length, and minimum width, respectively. Finally, if the user's selection is column-building, then the algorithm selects the addition point with minimum length, minimum width, and minimum height, respectively.

When Algorithm 6 loads item of type $i \in I$ to the best fit addition point, we add three new addition points to the set of addition points $P(k)$ if added item is not *fragile*. We set the coordinates of three new addition points as $(opposite_x, origin_y, origin_z)$, $(origin_x, opposite_y, origin_z)$, and $(origin_x, origin_y, opposite_z)$. Next, we delete the addition point where the item of type $i \in I$ is placed. For each container $k \in C$, the solution is also updated by adding the $volume(i)$ and $weight(i)$ of loaded item to $container_volume_packed(k)$ and $container_weight_packed(k)$, respectively. Similarly, total volume and weight packed for the incumbent solution is also updated by adding the $volume(i)$ and $weight(i)$ of loaded item to $incumbent_volume_packed$ and $incumbent_weight_packed$. The profit of the incumbent is calculated as follows:

$$incumbent_profit = \begin{cases} incumbent_profit + profit(i) - container_cost(k) & \text{if } |I(k)| = 1, \\ incumbent_profit + profit(i) & \text{if } |I(k)| > 1, \end{cases}$$

where $I(k)$ is the set of items placed in the container $k \in C$ so far.

4.2.2. Sorting the containers

Sort Containers (Algorithm 5) in Appendix B determines the order of containers to fill. If a generated random number is greater than or equal to the threshold value (τ), then Algorithm 5 orders the containers randomly. Otherwise, Algorithm 5 first sorts the containers in decreasing order of containers' priority. For the containers with equal priority, Algorithm 5 sorts them in descending order by volume packed of the containers in a given solution. If the containers that "may be packed" share the same amount of volume packed, then Algorithm 5 sorts them in decreasing order of the ratio of cost to volume capacity of the container.

4.3. Improvement heuristic

The Improvement Heuristic starts with setting the incumbent solution to best_known solution with a probability of $\lambda = 0.65$. Next, Improvement Heuristic randomly perturbs the order of rotations and the order of items in the incumbent solution to diversify the search. For each item type and each orientation o , we generate a random number. If the random number is less than 0.2, then we swap the order of orientation o with the order of another orientation selected randomly. Similarly, we generate another random number

Table 3
Improvement over constructive heuristic.

Instance	Constructive heuristic with			LNS w. Wall B.	
	Column B.	Layer B.	Wall B.	Min	Avg
IMM1	25	25	25	25	25
IMM2	10	10	10	10	10
IMM3	21	21	21	19	19
IMM4	28	28	28	26	26
IMM5	52	52	52	51	51
IMM6	10	10	10	10	10
IMM7	16	16	16	16	16
IMM8	4	4	4	4	4
IMM9	24	21	21	19	19
IMM10	55	55	55	55	55
IMM11	22	22	22	17	17
IMM12	53	53	53	53	53
IMM13	40	40	40	25	25
IMM14	29	29	29	27	27
IMM15	12	11	11	11	11
IMM16	29	26	26	26	26
IMM17	8	8	8	7	7
IMM18	3	3	3	2	2
IMM19	3	3	3	3	3
IMM20	6	5	5	5	5
IMM21	25	25	25	20	20.03
IMM22	10	10	9	8	8.97
IMM23	23	23	23	20	20.97
IMM24	7	7	7	5	5.67
IMM25	5	5	5	5	5
IMM26	4	4	4	3	3
IMM27	6	5	5	5	5
IMM28	13	10	10	10	10
IMM29	20	20	19	17	17
IMM30	28	24	24	22	22
IMM31	16	14	14	13	13
IMM32	4	4	4	4	4
IMM33	5	5	5	4	4.97
IMM34	9	9	9	8	8
IMM35	3	3	3	2	2
IMM36	14	14	14	14	14
IMM37	23	23	23	23	23
IMM38	45	45	45	45	45
IMM39	16	16	16	15	15
IMM40	9	9	9	8	8.94
IMM41	16	16	16	15	15
IMM42	6	5	5	4	4
IMM43	4	4	3	3	3
IMM44	4	4	4	4	4
IMM45	3	3	3	3	3
IMM46	2	2	2	2	2
IMM47	4	4	4	3	3
Total	774	755	752	696	700.54

for each item type $i \in I$. If this random number is less than 0.1, we swap the order of item type i with the order of another item type selected randomly. For each container $k \in C$, Improvement Heuristic destroys the incumbent solution either by emptying the container with a probability of $(1 - \text{container_volume_packed}(k) / \text{container_volume_capacity}(k)) / 2$ or removing a random subset of items from the container. If the incumbent solution is infeasible and item of type i is not mandatory to be packed, we remove the item of type i . Otherwise, we randomly determine the whether to remove the item of type i . When some items are removed from the container $k \in C$, Algorithm 3 also removes the remaining items from the container $k \in C$ since the remaining items might be unstable or vertical and optional front-side support might not be ensured. To satisfy the feasibility conditions, remaining items are packed into the container $k \in C$ using the AddItemToContainer(item of type i , container k). Algorithm 3 also calls Constructive Heuristic to load the unpacked items into the containers. The aim of this destruction is to find better solutions by rearranging the items. Half of CPU_time_limit set by the user is spent to repeatedly destruct and construct the solution.

After the destruction and construction steps, to compare the quality of the best_known solution with the quality of incumbent solution, Improvement Heuristic calculates the *best_known_distance* and *best_known_moment*, if the option *optimise_balance* is set to *TRUE*.

Next, for each container $k \in C$, Improvement Heuristic repeats the destruction and construction steps to increase the diversification on the incumbent solution until the time limit set by the user is reached. In other words, the remaining time is used for reorganising the containers a time, with equal time allocated to each non-empty container. If one of the following conditions hold:

1. *incumbent_feasible* = *TRUE* and *best_known_feasible* = *FALSE*
2. *incumbent_feasible* = *FALSE* and *best_known_feasible* = *FALSE* and
 $\text{incumbent_volume_packed} > \text{best_known_volume_packed}$
3. *incumbent_feasible* = *TRUE* and *best_known_feasible* = *TRUE* and
 $\text{incumbent_profit} > \text{best_known_profit}$
4. *incumbent_feasible* = *TRUE* and *best_known_feasible* = *TRUE* and
 $\text{incumbent_profit} = \text{best_known_profit}$ and
 $\text{incumbent_volume_packed} > \text{best_known_volume_packed}$
5. *incumbent_feasible* = *TRUE* and *best_known_feasible* = *TRUE* and
 $\text{incumbent_profit} = \text{best_known_profit}$ and
 $\text{incumbent_volume_packed} = \text{best_known_volume_packed}$ and
 $\text{incumbent_moment} < \text{best_known_moment}$
6. *incumbent_feasible* = *TRUE* and *best_known_feasible* = *TRUE* and
 $\text{incumbent_profit} = \text{best_known_profit}$ and
 $\text{incumbent_volume_packed} = \text{best_known_volume_packed}$ and
 $\text{incumbent_moment} = \text{best_known_moment}$ and
 $\text{incumbent_distance} < \text{best_known_distance}$

the Improvement Heuristic 3 sets the best_known solution to the incumbent solution.

4.4. Solution visualisation

Fig. 6 shows a snapshot of visualisation of a packing solution in 3D. It contains semi-transparent cuboids depicting the containers and the items packed into the containers. Users can also animate the loading plan. Furthermore, users can move the shapes around to see if they can arrive at a better solution.

5. Computational experiments

We now provide comparisons between our LNS algorithm and various algorithms from the literature based on their performance on benchmark data sets for both input minimisation and output maximisation problems. We have implemented LNS algorithm using Visual Basic Application into CLP Spreadsheet Solver. All experiments are performed on a computer with a 1.60 GHz - 1.80 GHz Intel R Core TM CPU i5-8265U and 16 GB main memory. We give the list of papers that we compare our results against in Table 2.

5.1. Results for input minimisation problem

We use the data set IMM proposed by Ivancic et al. (1989). The instance set contains 47 instances with homogeneous containers, with each instance using a different container size. The number of item types ranges from two to five and total number of items ranges from 47 to

Table 4
Best known results for IMM instances.

Instance	I.	B.R.	B.	E.	L.Z.	T.	C.	L.	Z.	Z. with	C.S.	K.S.J.S.C.	K.S.J.S.C. with	LNS Alg. with VS	
	(1989)	(1995)	(2000)	(2003)	(2005)	(2008)	(2011)	(2012)	(2012)	VS (2012)	(2013)	(2020)	VS (2020)	(min)	(avg)
IMM1	26	27	25	25	25	25	25	25	25	25	25	25	25	25	25
IMM2	11	11	10	10	10	10	10	10	10	10	10	9	10	10	10
IMM3	20	21	20	20	19	20	19	19	19	19	19	19	19	19	19
IMM4	27	29	28	26	26	26	26	26	26	26	26	26	26	26	26
IMM5	65	61	51	51	51	51	51	51	51	51	51	51	51	51	51
IMM6	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10
IMM7	16	16	16	16	16	16	16	16	16	16	16	16	16	16	16
IMM8	5	4	4	4	4	4	4	4	4	4	4	4	4	4	4
IMM9	19	19	19	19	19	19	19	19	19	19	19	19	19	19	19
IMM10	55	55	55	55	55	55	55	55	55	55	55	55	55	55	55
IMM11	18	19	18	17	16	16	16	16	16	17	17	16	16	17	17
IMM12	55	55	53	53	53	53	53	53	53	53	53	53	53	53	53
IMM13	27	25	25	25	25	25	25	25	25	25	25	25	25	25	25
IMM14	28	27	28	27	27	27	27	27	27	27	27	27	27	27	27
IMM15	11	11	11	11	11	11	11	11	11	11	11	11	11	11	11
IMM16	34	28	26	26	26	26	26	26	26	26	26	26	26	26	26
IMM17	8	8	7	7	7	7	7	7	7	7	7	7	7	7	7
IMM18	3	3	2	2	2	2	2	2	2	2	2	2	2	2	2
IMM19	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
IMM20	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5
IMM21	24	24	21	20	20	20	20	20	20	20	20	20	20	20	20.03
IMM22	10	11	9	8	9	9	8	9	8	8	8	8	8	8	8.97
IMM23	21	22	20	20	20	20	19	20	19	20	20	19	21	20	20.97
IMM24	6	6	6	6	5	5	5	5	5	5	6	5	5	5	5.67

Table 5
Best known results for IMM instances.

Instance	I.	B.R.	B.	E.	L.Z.	T.	C.	L.	Z.	Z. (2012)	C.S.	K.S.J.S.C.	K.S.J.S.C. (2020)	LNS Alg. with VS	
	(1989)	(1995)	(2000)	(2003)	(2005)	(2008)	(2011)	(2012)	(2012)	with VS	(2013)	(2020)	with VS	(min)	(avg)
IMM25	6	5	5	5	5	5	5	5	5	5	5	5	5	5	5
IMM26	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
IMM27	5	5	5	5	5	5	5	5	4	4	5	4	4	5	5
IMM28	10	11	10	10	9	10	10	9	10	10	10	9	9	10	10
IMM29	18	17	17	17	17	17	17	17	17	17	17	16	17	17	17
IMM30	24	24	22	22	22	22	22	22	22	22	22	22	22	22	22
IMM31	13	13	13	13	12	13	12	12	12	12	13	13	13	13	13
IMM32	5	4	4	4	4	4	4	4	4	4	4	4	4	4	4
IMM33	5	5	5	5	4	5	4	4	4	4	4	4	4	4	4.97
IMM34	9	9	8	8	8	8	8	8	8	8	8	8	8	8	8
IMM35	3	3	2	2	2	2	2	2	2	2	2	2	2	2	2
IMM36	18	19	14	14	14	14	14	14	14	14	14	14	14	14	14
IMM37	26	27	23	23	23	23	23	23	23	23	23	23	23	23	23
IMM38	50	56	45	45	45	45	45	45	45	45	45	45	45	45	45
IMM39	16	16	15	15	15	15	15	15	15	15	15	15	15	15	15
IMM40	9	10	9	8	9	9	8	9	8	8	8	8	8	8	8.94
IMM41	16	16	15	15	15	15	15	15	15	15	15	15	15	15	15
IMM42	4	5	4	4	4	4	4	4	4	4	4	4	4	4	4
IMM43	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
IMM44	4	4	3	4	3	3	3	3	3	3	3	3	3	4	4
IMM45	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
IMM46	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
IMM47	4	3	3	3	3	3	3	3	3	3	3	3	3	3	3
Total	763	763	705	699	694	698	692	694	691	693	696	689	693	696	700.54

180. All items “must be packed”. The objective of the problem is to minimise the number of containers used (i.e., cost of each container is equal to 1.) IMM covers only a small subset of features of the CLP, since there is no weight, rotation, stacking, incompatibility, load balance, or front side support constraint.

We report in Tables 3, 4, 5, and 6 the number of containers used in the best solution found by Constructive Heuristics (column building, layer building, and wall building) and the LNS Algorithm, along with the results from the literature. We highlight the best known solutions in bold. In order to say that there is no significantly randomisation effect on the performance of our LNS algorithm, we run it 30 times by using different random numbers and report the minimum and average of the results. We also set *CPU_time_limit* to three minutes for each instance.

There is no definitive way of telling which of the constructive heuristics will work better for a given problem instance, users will

need to try them out. On IMM instances, we test the performance of constructive heuristics by turning off the improvement phase of the LNS algorithm. We solve the instances by setting the constructive heuristic option to column, layer and wall-building separately, and then report their results in Table 3. The results show that Constructive Heuristic with wall-building heuristic works better than column and layer-building heuristics. Therefore, we execute our LNS Algorithm by setting the option of Constructive Heuristic to wall-building in the computational experiments. It is evident from Table 3 that Improvement Heuristic of our LNS Algorithm improves the results of the construction phase with wall-building heuristics by 8% on the average.

Our LNS algorithm has been able to obtain better results than many of the algorithms in the literature. It is clear that our algorithm 1 perform significantly better than the original work Ivancic et al. (1989)

Table 6
Best known results for IMM instances (with separation of items).

Instance	E. (2003)	K.S.J.S.C. (2020)	K.S.J.S.C. (2020) w. vertical support	LNS Alg with vertical support			
				& horizontal support			
				Min	Avg	Min	Avg
IMM1	27	27	27	27	27	27	27
IMM2	11	10	10	11	11	11	11
IMM3	20	20	20	20	20.83	21	21.97
IMM4	28	28	28	28	28	28	28
IMM5	51	51	51	51	51	53	53
IMM6	10	10	10	10	10	10	10
IMM7	16	16	16	16	16	16	16
IMM8	4	4	4	4	4	4	4
IMM9	19	19	19	19	19	19	19
IMM10	55	55	55	55	55	55	55
IMM11	18	17	18	18	18	19	19
IMM12	56	56	56	56	56	56	56
IMM13	25	25	25	25	25	25	25
IMM14	28	28	28	28	28	28	28
IMM15	12	12	12	12	12	12	12
IMM16	26	26	26	26	26	26	26
IMM17	9	9	9	9	9	9	9
IMM18	2	2	2	2	2	2	2
IMM19	3	3	3	3	3	3	3
IMM20	5	5	5	5	5	5	5
IMM21	21	20	20	20	20.56	21	21.1
IMM22	8	8	8	9	9	9	9.2
IMM23	21	20	21	21	21	21	21
IMM24	6	5	5	5	5.73	6	6
IMM25	5	5	5	5	5	5	5
IMM26	3	3	3	3	3	3	3
IMM27	5	5	5	5	5	5	5
IMM28	10	9	10	10	10	10	10.23
IMM29	17	16	17	17	17	17	17
IMM30	23	22	23	23	23	23	23
IMM31	13	13	13	13	13	13	13
IMM32	4	4	4	4	4	4	4
IMM33	5	5	5	5	5	5	5
IMM34	9	8	8	8	8	9	9
IMM35	3	3	3	3	3	3	3
IMM36	18	18	18	19	19	19	19
IMM37	23	23	23	23	23	23	23
IMM38	45	45	45	45	45	45	45
IMM39	15	15	15	15	15	15	15
IMM40	9	9	9	9	9	9	9
IMM41	16	16	16	16	16	16	16
IMM42	5	4	4	4	4.6	4	4.57
IMM43	4	3	3	3	3	3	3.67
IMM44	4	4	4	4	4	4	4
IMM45	3	3	3	3	3	3	3
IMM46	2	2	2	2	2	2	2
IMM47	3	4	4	4	4	4	4
Sum:	725	715	720	723	725.72	730	732.74

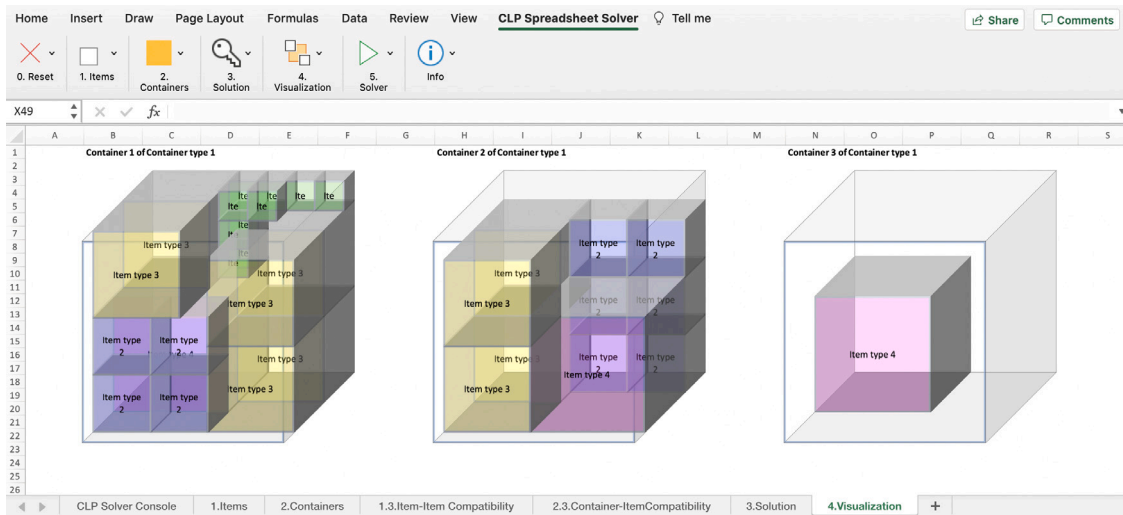
Table 7
Best known results for LN instances.

Instance	B.M.	N.	B.R.	G.B.	B.G.	B.G.	E.	L.	M.O.	W.	H.H.	D.D.	R.	L.	T.E.W.B.	LNS	
	(1990)	(1994)	(1995)	(1997)	(1998)	(2001)	(2002)	(2005)	(2005)	(2008)	(2010)	(2011)	(2011)	(2012)	(2017)	Avg	Max
LN1	62.5	62.5	62.5	62.5	62.5	62.5	62.5	62.5	62.5	62.5	62.5	62.5	62.5	62.5	62.5	62.5	62.5
LN2	90	80.7	90	89.5	96.6	89.8	90.8	80.4	92.6	90.7	97.9	86.3	97.9	96.4	93.2	91.8	94.6
LN3	53.4	53.4	53.4	53.4	53.4	53.4	53.4	53.4	53.4	53.4	53.4	53.4	53.4	53.4	53.4	53.4	53.4
LN4	55	55	55	55	55	55	55	55	55	55	55	55	55	55	55	55	55
LN5	77.2	77.2	77.2	77.2	77.2	77.2	77.2	76.7	77.2	77.2	77.2	77.2	77.2	77.2	77.2	77.2	77.2
LN6	83.1	88.7	83.1	91.1	91.2	92.4	87.9	84.8	91.7	92.9	96.7	89.2	96.3	93.5	93	89.8	92.6
LN7	78.7	81.8	78.7	83.3	84.7	84.7	84.7	77	84.7	84.7	84.7	83.2	84.7	84.7	84.7	84.5	84.7
LN8	59.4	59.4	59.4	59.4	59.4	59.4	59.4	59.4	59.4	59.4	59.4	59.4	59.4	59.4	59.4	59.4	59.4
LN9	61.9	61.9	61.9	61.9	61.9	61.9	61.9	61.9	61.9	61.9	61.9	61.9	61.9	61.9	61.9	61.9	61.9
LN10	67.3	67.3	67.3	67.3	67.3	67.3	67.3	67.3	67.3	67.3	67.3	67.3	67.3	67.3	67.3	67.3	67.3
LN11	62.2	62.2	62.2	62.2	62.2	62.2	62.2	62.2	62.2	62.2	62.2	62.2	62.2	62.2	62.2	62.2	62.2
LN12	78.5	78.5	78.5	78.5	78.5	78.5	78.5	69.5	78.5	78.5	78.5	78.5	78.5	78.5	78.5	78.5	78.5
LN13	78.1	84.1	78.1	85.6	84.3	85.6	85.6	73.3	85.6	85.6	85.6	85.6	85.6	84.9	85.6	85.4	85.6
LN14	62.8	62.8	62.8	62.8	62.8	62.8	62.8	62.8	62.8	62.8	62.8	62.8	62.8	62.8	62.8	62.8	62.8
LN15	59.5	59.5	59.5	59.5	59.5	59.5	59.5	59.5	59.5	59.5	59.5	59.5	59.5	59.5	59.5	59.5	59.5
Avg.	68.64	69	68.6	69.9	70.4	70.1	69.91	67	70.3	70.2	71.0	69.55	70.93	70.6	70.4	70.1	70.5

Table 8

Our best found results for BR instances with side constraints.

Instances	Output maximisation		Input minimisation	
	Avg vol. util. (%)	Max vol. util. (%)	Avg # of cont.	Min # of cont.
BR1	86.3	92.2	2.00	2
BR2	81.7	88.4	2.00	2
BR3	78.6	88.8	2.11	2
BR4	70.5	87.0	2.30	2
BR5	70.9	84.5	2.50	2
BR6	64.5	78.3	2.93	2
BR7	60.3	72.7	3.05	3
BR8	50.3	62.1	3.81	3
BR9	49.1	61.1	4.00	4
BR10	46.6	56.3	4.11	3
BR11	41.6	53.2	4.25	4
BR12	39.8	53.2	4.42	4
BR13	36.2	45.6	4.81	4
BR14	33.9	44.0	4.97	4
BR15	33.4	44.1	4.95	4

**Fig. 6.** A snapshot of visualisation worksheet.

and Bischoff and Ratcliff (1995). We also improve upon the later works by Bortfeldt (2000), Eley (2003) and Takahara (2008). It is important to note that our LNS algorithm guarantees full vertical stability by supporting the bottom of items by either the container floor or by the items underneath. Our results with vertical stability are quite close to the more recent works of Zhu et al. (2012), Ceschia and Schaerf (2013) and Kurpel et al. (2020) in which full vertical stability is ensured. Although the LNS algorithm requires an additional container for the instances IMM11, IMM27, IMM28, and IMM44 compared to the results of Kurpel et al. (2020), our LNS algorithm performs better than the work of Kurpel et al. (2020) for the instance IMM23, thereby difference concerning the total number of containers used is only 3 (0.4%).

We also consider the situation with the separation of items as well as with the consideration of vertical stability. As examined in Eley (2003), we imposed that items of the first two types must not be stowed in the same container. The results in Table 6 indicates that our LNS algorithm outperforms the procedure of Eley (2003). Compared to the work of Kurpel et al. (2020) which considers both separation of items and vertical stability, only for IMM2, IMM22, and IMM36, we require an additional container. Finally, we solve the IMM test instances by

including the horizontal support constraint together with the separation of items and vertical stability. Since we are the first who solves the IMM instances with these additional constraints, no benchmark exists, and we provide our results in the last two columns of Table 6.

5.2. Results for output maximisation problem

In the computational experiments, we also take the single container problem where the aim of the problem is to maximise the volume utilisation of the container. We adopt our LNS algorithm to this problem setting by setting the profit of each item to its volume and the cost of the container to zero. We also set the priority of each item to “may be packed”.

We use the LN1–LN15 instances introduced by Loh and Nee (1992) to compare the results of our LNS algorithm with the state of the art. In these instances, rotations around the horizontal axes are not allowed. The instance set contains 100 to 250 items with 6 to 10 different types.

To solve each instance, we execute the LNS algorithm for 10 min. In order to eliminate the randomisation effect, we run the LNS algorithm 30 times for each instance. We again highlight the best results in bold

Algorithm 3: Improvement Heuristic

Input: Item, container, and compatibility data, incumbent solution, best_known solution, λ , γ

while $CPU_time_limit/2$ is not exceeded **do**

$Rnd \leftarrow U[0,1]$

if $Rnd < \lambda$ **then**

$incumbent \leftarrow best_known$

Randomly perturb rotation and order of items in the incumbent solution

for $k \in C$ **do**

$Rnd \leftarrow U[0,1]$

if $Rnd < (1 - container_volume_packed(k) / container_volume_capacity(k)) / 2$ **then**

Empty container k

else

Randomly remove a subset of items from container k

Repack remaining items in container k

end

Call Constructive Heuristic to pack removed items

end

Set $incumbent_distance$, $best_known_distance$, $incumbent_moment$, $best_known_moment$ to 0

for $k \in C$ **do**

Calculate $distance(k)$ using equation (5)

$best_known_distance := best_known_distance + distance(k)$

if $optimise_balance = TRUE$ **then**

Calculate $moment(k)$ using equation (4)

$best_known_moment := best_known_moment + moment(k)$

while $CPU_time_limit_foreorganise$ is not exceeded **do**

$Rnd \leftarrow U[0,1]$

if $Rnd < \gamma$ **then**

$incumbent \leftarrow best_known$

Randomly perturb rotation and order of items

if $Rnd < (1 - container_volume_packed(k) / container_volume_capacity(k)) / 2$ **then**

Empty container k

else

Randomly remove a subset of items from container k

Repack remaining items in container k

Call AddItemToContainer to pack removed items into the container

$k \in C$

$incumbent_feasible \leftarrow True$

for $i \in I$ **do**

if (There exists at least one item of type $i \in I$ to be packed) and

($mandatory(i) = TRUE$) **then**

$incumbent_feasible \leftarrow False$

exit for

end

Calculate $distance(k)$ using equation (5)

$incumbent_distance = incumbent_distance + distance(k)$

if $optimise_balance = TRUE$ **then**

Calculate $moment(k)$ using equation (4)

$incumbent_moment = incumbent_moment + moment(k)$

if $incumbent$ solution is better than the $best_known$ solution **then**

$best_known \leftarrow incumbent$

end

end

Output: best_known solution

in Table 7. Table 7 indicates that our LNS algorithm is very competitive with the state of the art for those instances. We obtain the optimality for 13/15 of the instances (all items are loaded in a single container). For the remaining two instances, our LNS algorithm achieves 94.6% and 92.6% volume utilisation, where the best known results are 97.9% and 96.7%, respectively.

5.3. Results for BR instances

To test the broader application range of LNS algorithm, we generate a set of instances based on the single container loading instances (known as BR instances) introduced by Bischoff and Ratcliff (1995) and Davies and Bischoff (1999). The original data includes 15 groups of 100 instances. The first seven groups include instances with weakly heterogeneous items, i.e. the number of item types varies from 3 to 20, while, the last eight groups contain instances with strongly heterogeneous items, i.e., the number of item types varies from 30 to 100. In the computational experiments, we only use the first 10 instances in each group. We organised the first 5 of 10 instances in each group as output maximisation problems and converted the last 5 of 10 instances in each

group as input minimisation problems by setting each item “must be packed”.

The original instances already include orientation constraints. To show the ability of LNS algorithm to handle the practical constraints, we include additional side constraints. We minimise the moment of the item weights along the central z-axis to achieve an even weight distribution. We also ensure the vertical and horizontal stability. For the input minimisation problems, we additionally include the separation constraints such that the items of the first two types must not be loaded into the same container.

We execute the LNS algorithm for three minutes to solve each instance and run the algorithm 30 times by using different random numbers. In Table 8, for output maximisation problems, we report the average and maximum volume utilisations of the container in the five instances of each group and for the input minimisation problems, we report the average and minimum number of containers required in the five instances of each group. Since we do not allow to load the two item types into the same container, the number of containers required to load all items is at least 2 for each instance. As the number of item types increases, the volume utilisation of the container decreases for output maximisation problems and the number of containers required to load all items increases for the input minimisation problems. We believe these generated instances with several side constraints of CLP can be utilised as benchmark instances.

6. Conclusion

We have a new LNS algorithm to efficiently determine the packing of items into the containers. The main advantage of our LNS algorithm is that it is generally applicable to solve both input minimisation and output maximisation types of CLPs with different degree of heterogeneity of the items (likewise containers). Besides, our LNS algorithm can accommodate many side constraints imposed by the users such as weight limit, loading priorities, orientation, stacking, separation, and stability. Moreover, our LNS algorithm attempts to achieve an even weight distribution through minimising the moment of the item weights along the central z-axis. We believe that the VBA implementation of our algorithm, CLP Spreadsheet Solver, is suitable to be used by small and medium-sized enterprises as well as for teaching purposes.

There are some future research directions arising from this work. The performance of our LNS algorithm depends on a strategy that destructs and perturbs the current solution. It is still an open question how to select the strategy which eventually yields better solutions. Furthermore, the effectiveness of the proposed algorithm could enable the researchers to accommodate more practical constraints to meet the requirements of other users, e.g., order of unloading must be taken into account when a truck with multiple stops is being loaded.

CRedit authorship contribution statement

Özge Şafak: Data curation, Methodology, Visualization, Roles/Writing – original draft, Writing – review & editing. **Güneş Erdoğan:** Methodology, Software, Roles/Writing – original draft, Writing – review & editing.

Data availability

Data will be made available on request.

Appendix A. Sorting items

See Algorithm 4.

Appendix B. Sorting containers

See Algorithm 5.

Algorithm 4: Sort Items

Input: item data
 $item(i), i \in I \leftarrow$ list of item types
Sort the $item(i), i \in I$ with respect to their priorities
 $S \leftarrow$ set of item types with $mandatory(i) = TRUE, i \in I$
if $item_sort_criterion = "volume"$ **then**
Sort the $item(i), i \in S$ in decreasing order of
 $sort_criterion(i) := volume(i) \times max_weight + weight(i), i \in S,$
where $max_weight := \max_{i \in I} \{weight(i)\} + 1$
else if $item_sort_criterion = "weight"$ **then**
Sort the $item(i), i \in S$ in decreasing order of
 $sort_criterion(i) := weight(i) \times max_volume + volume(i), i \in S,$
where $max_volume := \max_{i \in I} \{volume(i)\} + 1$
else
 $item_max(i) := \max\{width(i), height(i), length(i)\}, i \in S,$
Sort the $item(i), i \in S$ in decreasing order of
 $sort_criterion(i) := item_max(i) \times max_volume + volume(i), i \in S,$
where $max_volume := \max_{i \in I} \{volume(i)\} + 1$
 $\bar{S} \leftarrow$ set of item types with $mandatory(i) = FALSE, i \in I$
Sort the $item(i), i \in \bar{S}$ in decreasing order of $ratio(i) := profit(i) / volume(i), i \in \bar{S}$
Output: $item(i), i \in I$

Algorithm 5: Sort Containers

Input: container data, solution, τ
 $C \leftarrow$ set of containers
 $S' \leftarrow$ set of containers such that $container_mandatory(k) = TRUE, k \in C$
 $S'' \leftarrow$ set of containers such that $container_mandatory(k) = FALSE, k \in C$
 $container(k), k \in C \leftarrow$ list of containers in given solution
 $container_volume_packed(k) \leftarrow$ volume packed of the container $k \in C$ in given solution
 $Rnd \leftarrow U[0,1]$
if $Rnd < \tau$ **then**
Sort $container(k), k \in C$ with respect to their priority
Sort $container(k)$ in decreasing order of $container_volume_packed(k), k \in S'$
Sort $container(k)$ in decreasing order of $container_volume_packed(k), k \in S''$
for $m \in S''$ **do**
for $k \in S'', s.t. m \neq k$ **do**
if $container_volume_packed(m) = container_volume_packed(k)$ **then**
if $container_cost(m) / container_volume_capacity(m) \leq$
 $container_cost(k) / container_volume_capacity(k)$ **then**
Prioritise $container(m)$ above $container(k)$
end
end
else
Randomly reorder $container(k), k \in C$
Output: $container(k), k \in C$

Appendix C. Formulas

The moment ($moment(m)$) of the container m is calculated as follows:

$$| \sum_{j \in I(m)} (item_origin_x(j) + item_opposite_x(j) - container_width(m)) \times weight(t(j)) |$$

$$2$$
(4)

where $I(m)$ is the set of items packed in the container m in the incumbent solution and $t(j)$ is the type of item $j \in I(m)$. $item_origin_x(j)$ and $item_opposite_x(j)$ are the x coordinates of corners of item j . $container_width(m)$ and is the width (x) of the container. $weight(t(j))$ is the weight of the item of type $t(j)$.

Algorithm 2 calculates the distances between the items of same type in each container $m \in C$ as follows:

$$distance(m) = \sum_{j,s \in I(m) | t(j)=t(s)} |item_opposite_x(j) + item_origin_x(j) - item_opposite_x(s) - item_origin_x(s)|$$

$$+ |item_opposite_y(j) + item_origin_y(j) - item_opposite_y(s) - item_origin_y(s)|$$

$$+ |item_opposite_z(j) + item_origin_z(j) - item_opposite_z(s) - item_origin_z(s)|$$

$$+ \sum_{j \in I(m)} item_opposite_z(j) + (|I(m)|)^2 \cdot max_z,$$
(5)

where $max_z = \max_{j \in I(m)} item_opposite_z(j)$.

Appendix D. Packing an item to a container**Algorithm 6: AddItemToContainer(item of type i , container k)**

Input: Item, container, and compatibility data
Set min_x, min_y , and min_z to container's width, height, and length, respectively
if feasibility checks listed in Section 4.2.1 fails **then**
| **Finish** AddItemToContainer(item of type i , container k)
end
for $r \in O(i)$ **do**
if orientation r for the item of type i is forbidden by the user **then**
| **Go to next** r
end
for $p \in P$ **do**
Set $origin_x, origin_y, origin_z$ to the coordinates of the addition point p
if item of type i is a heavy item & $origin_y > 0$ **then**
| **Go to next** p
end
Set $opposite_x, opposite_y$ and $opposite_z$ for the orientation r as in Equation (3)
if ($opposite_x > container_width(k)$) or ($opposite_y > container_height(k)$) or ($opposite_z > container_length(k)$) **then**
| **Go to next** p
end
if feasibility check for all corners w.r.t to the other items fails **then**
| **Go to next** p
end
if $origin_y > 0$ **then**
| **if** intersection area of the item of type i and items directly below it less than the bottom area of the item of type i **then**
| | **Go to next** p
| **end**
end
if front_side_support **then**
| **if** $origin_y > 0$ **then**
| | **if** intersection area of the item of type i and items directly below it less than the bottom area of the item of type i **then**
| | | **Go to next** p
| | **end**
| **end**
end
if constructive_heuristic == wall-building **then**
| **if** $origin_z < min_z$ OR ($origin_z = min_z$ & $origin_y < min_y$) OR ($origin_z = min_z$ & $origin_y = min_y$ & $origin_x < min_x$) **then**
| | $min_x := origin_x, min_y := origin_y, min_z := origin_z$
| | candidate_position := p
| **end**
| **else if** constructive_heuristic == layer-building **then**
| **if** $origin_y < min_y$ OR ($origin_y = min_y$ & $origin_z < min_z$) OR ($origin_y = min_y$ & $origin_z = min_z$ & $origin_x < min_x$) **then**
| | $min_x := origin_x, min_y := origin_y, min_z := origin_z$
| | candidate_position := p
| **end**
| **else**
| **if** $origin_z < min_z$ OR ($origin_z = min_z$ & $origin_x < min_x$) OR ($origin_z = min_z$ & $origin_x = min_x$ & $origin_y < min_y$) **then**
| | $min_x := origin_x, min_y := origin_y, min_z := origin_z$
| | candidate_position := p
| **end**
| **end**
end
end
if candidate_position is null **then**
| $IsItemAdded := FALSE$
end
else
 $IsItemAdded := TRUE$
if item of type i is not fragile **then**
| $P(k) := P(k) \cup \{(opposite_x, origin_y, origin_z), (origin_x, opposite_y, origin_z), (origin_x, origin_y, opposite_z)\}$
end
 $P(k) := P(k) \setminus p$
Set the coordinates of added item
Update the $incumbent_volume_packed, incumbent_weight_packed,$ and $incumbent_profit$
end
Return: $IsItemAdded$

References

- Ali, S., Ramos, A.G., Carravilla, M.A., Oliveira, J.F., 2022. On-line three-dimensional packing problems: a review of off-line and on-line solution approaches. *Comput. Ind. Eng.* 108122.
- Bischoff, E.E., Marriot, M.D., 1990. A comparative evaluation of heuristics for container loading. *European J. Oper. Res.* 44 (2), 267–276.
- Bischoff, E.E., Ratcliff, M.S.W., 1995. Issues in the development of approaches to container loading. *Omega* 23 (4), 377–390.
- Bortfeldt, A., 2000. Eine heuristik für multiple containerladeprobleme. *OR-Spektrum* 22 (2), 239–261.
- Bortfeldt, A., Gehring, H., 1998. Applying tabu search to container loading problems. In: *Operations Research Proceedings 1997*. Springer, pp. 533–538.
- Bortfeldt, A., Gehring, H., 2001. A hybrid genetic algorithm for the container loading problem. *European J. Oper. Res.* 131 (1), 143–161.
- Bortfeldt, A., Wäscher, G., 2013. Constraints in container loading—A state-of-the-art review. *European J. Oper. Res.* 229 (1), 1–20.
- Bowes, P., 2021. Pitney bowes parcel shipping index. <https://www.pitneybowes.com/us/shipping-index.html>, Accessed November 2021.
- Ceschia, S., Schaefer, A., 2013. Local search for a multi-drop multi-container loading problem. *J. Heuristics* 19 (2), 275–294.
- Chan, F.T., Bhagwat, R., Kumar, N., Tiwari, M., Lam, P., 2006. Development of a decision support system for air-cargo pallets loading problem: A case study. *Expert Syst. Appl.* 31 (3), 472–485.
- Che, C.H., Huang, W., Lim, A., Zhu, W., 2011. The multiple container loading cost minimization problem. *European J. Oper. Res.* 214 (3), 501–511.
- Chen, C., Lee, S., Shen, Q., 1995. An analytical model for the container loading problem. *European J. Oper. Res.* 80 (1), 68–76.
- Chien, C., Deng, J., 2004. A container packing support system for determining and visualizing container packing patterns. *Decis. Support Syst.* 37 (1), 23–34.
- Correcher, J.F., Alonso, M.T., Parreño, F., Alvarez-Valdés, R., 2017. Solving a large multicontainer loading problem in the car manufacturing industry. *Comput. Oper. Res.* 82, 139–152.
- Davies, A.P., Bischoff, E.E., 1999. Weight distribution considerations in container loading. *European J. Oper. Res.* 114 (3), 509–527.
- Dereli, T., Daş, G.S., 2010. Development of a decision support system for solving container loading problems. *Transport* 25 (2), 138–147.
- Dereli, T., Daş, G.S., 2011. A hybrid ‘bee (s) algorithm’ for solving container loading problems. *Appl. Soft Comput.* 11 (2), 2854–2862.
- Egeblad, J., Garavelli, C., Lisi, S., Pisinger, D., 2010. Heuristics for container loading of furniture. *European J. Oper. Res.* 200 (3), 881–892.
- Eley, M., 2002. Solving container loading problems by block arrangement. *European J. Oper. Res.* 141 (2), 393–409.
- Eley, M., 2003. A bottleneck assignment approach to the multiple container loading problem. *OR Spectrum* 25 (1), 45–60.
- Fasano, G., 2004. A MIP approach for some practical packing problems: Balancing constraints and tetris-like items. *Q. J. Belgian French Ital. Oper. Res. Soc.* 2 (2), 161–174.
- Fuellerer, G., Doerner, K.F., Hartl, R.F., Iori, M., 2010. Metaheuristics for vehicle routing problems with three-dimensional loading constraints. *European J. Oper. Res.* 201 (3), 751–759.
- Gehring, H., Bortfeldt, A., 1997. A genetic algorithm for solving the container loading problem. *Int. Trans. Oper. Res.* 4 (5–6), 401–418.
- Gendreau, M., Iori, M., Laporte, G., Martello, S., 2006. A tabu search algorithm for a routing and container loading problem. *Transp. Sci.* 40 (3), 342–350.
- George, J.A., Robinson, D.F., 1980. A heuristic for packing boxes into a container. *Comput. Oper. Res.* 7 (3), 147–156.
- He, K., Huang, W., 2010. A caving degree based flake arrangement approach for the container loading problem. *Comput. Ind. Eng.* 59 (2), 344–351.
- Hifi, M., Kacem, I., Nègre, S., Wu, L., 2010. A linear programming approach for the three-dimensional bin-packing problem. *Electron. Notes Discrete Math.* 36, 993–1000.
- Iori, M., Martello, S., 2010. Routing problems with loading constraints. *Top* 18 (1), 4–27.
- Ivancic, N., Mathur, K., Mohanty, B., 1989. An integer programming based heuristic approach to the three-dimensional packing problem. *J. Manuf. Oper. Manage.* 2 (4), 268–298.
- Júnior, R.R., Yanasse, H.H., Morabito, R., Junqueira, L., 2019. A hybrid approach for a multi-compartment container loading problem. *Expert Syst. Appl.* 137, 471–492.
- Junqueira, L., Morabito, R., Yamashita, D.S., 2012a. Three-dimensional container loading models with cargo stability and load bearing constraints. *Comput. Oper. Res.* 39 (1), 74–85.
- Junqueira, L., Morabito, R., Yamashita, D.S., 2012b. MIP-based approaches for the container loading problem with multi-drop constraints. *Ann. Oper. Res.* 199 (1), 51–75.
- Kurpel, D.V., Scarpin, C.T., Junior, J.E.P., Schenekemberg, C.M., Coelho, L.C., 2020. The exact solutions of several types of container loading problems. *European J. Oper. Res.* 284 (1), 87–107.
- Li, Y., Tang, X., Cai, W., 2014. On dynamic bin packing for resource allocation in the cloud. In: *Proceedings of the 26th ACM Symposium on Parallelism in Algorithms and Architectures*. pp. 2–11.
- Lim, A., Ma, H., Xu, J., Zhang, X., 2012. An iterated construction approach with dynamic prioritization for solving the container loading problems. *Expert Syst. Appl.* 39 (4), 4292–4305.
- Lim, A., Rodrigues, B., Yang, Y., 2005. 3-D container packing heuristics. *Appl. Intell.* 22 (2), 125–134.
- Lim, A., Zhang, X., 2005. The container loading problem. In: *Proceedings of the 2005 ACM Symposium on Applied Computing*. pp. 913–917.
- Liu, S., Tan, W., Xu, Z., Liu, X., 2014. A tree search algorithm for the container loading problem. *Comput. Ind. Eng.* 75, 20–30.
- Liu, J., Yue, Y., Dong, Z., Maple, C., Keech, M., 2011. A novel hybrid tabu search approach to container loading. *Comput. Oper. Res.* 38 (4), 797–807.
- Loh, T., Nee, A., 1992. A packing algorithm for hexahedral boxes. In: *Proceedings of the Conference of Industrial Automation, Singapore*. pp. 115–126.
- Mack, D., Bortfeldt, A., Gehring, H., 2004. A parallel hybrid local search algorithm for the container loading problem. *Int. Trans. Oper. Res.* 11 (5), 511–533.
- Moura, A., Oliveira, J.F., 2005. A GRASP approach to the container-loading problem. *IEEE Intell. Syst.* 20 (4), 50–57.
- do Nascimento, O.X., de Queiroz, T.A., Junqueira, L., 2021. Practical constraints in the container loading problem: Comprehensive formulations and exact algorithm. *Comput. Oper. Res.* 128, 105186.
- Ngoi, B., Tay, M., Chua, E., 1994. Applying spatial representation techniques to the container packing problem. *Int. J. Prod. Res.* 32 (1), 111–123.
- Paquay, C., Schyns, M., Limbourg, S., 2016. A mixed integer programming formulation for the three-dimensional bin packing problem deriving from an air cargo application. *Int. Trans. Oper. Res.* 23 (1–2), 187–213.
- Parreño, F., Alvarez-Valdés, R., Oliveira, J.F., Tamarit, J.M., 2010. Neighborhood structures for the container loading problem: a VNS implementation. *J. Heuristics* 16 (1), 1–22.
- Pisinger, D., 2002. Heuristics for the container loading problem. *European J. Oper. Res.* 141 (2), 382–392.
- Pollaris, H., Braekers, K., Caris, A., Janssens, G.K., Limbourg, S., 2016. Capacitated vehicle routing problem with sequence-based pallet loading and axle weight constraints. *EURO J. Transp. Logist.* 5 (2), 231–255.
- Ramos, A.G., Silva, E., Oliveira, J.F., 2018. A new load balance methodology for container loading problem in road transportation. *European J. Oper. Res.* 266 (3), 1140–1152.
- Ren, J., Tian, Y., Sawaragi, T., 2011. A tree search method for the container loading problem with shipment priority. *European J. Oper. Res.* 214 (3), 526–535.
- Sciomachen, A., Tanfani, E., 2007. A 3D-BPP approach for optimising stowage plans and terminal productivity. *European J. Oper. Res.* 183 (3), 1433–1446.
- Silva, E.F., Toffolo, T.A.M., Wauters, T., 2019. Exact methods for three-dimensional cutting and packing: A comparative study concerning single container problems. *Comput. Oper. Res.* 109, 12–27.
- Takahara, S., 2008. A multi-start local search approach to the multiple container loading problem. In: *Greedy Algorithms*. IntechOpen.
- Terno, J., Scheithauer, G., Sommerweiß, U., Riehme, J., 2000. An efficient approach for the multi-pallet loading problem. *European J. Oper. Res.* 123 (2), 372–381.
- Toffolo, T.A.M., Esprit, E., Wauters, T., Berghe, G.V., 2017. A two-dimensional heuristic decomposition approach to a three-dimensional multiple container loading problem. *European J. Oper. Res.* 257 (2), 526–538.
- Tsai, J., Wang, P., Lin, M., 2015. A global optimization approach for solving three-dimensional open dimension rectangular packing problems. *Optimization* 64 (12), 2601–2618.
- Wang, Z., Li, K.W., Levy, J.K., 2008. A heuristic for the container loading problem: A tertiary-tree-based dynamic space decomposition approach. *European J. Oper. Res.* 191 (1), 86–99.
- Wäscher, G., Haußner, H., Schumann, H., 2007. An improved typology of cutting and packing problems. *European J. Oper. Res.* 183 (3), 1109–1130.
- Wu, Y., Li, W., Goh, M., de Souza, R., 2010. Three-dimensional bin packing problem with variable bin height. *European J. Oper. Res.* 202 (2), 347–355.
- Zhang, D., Peng, Y., Leung, S.C., 2012. A heuristic block-loading algorithm based on multi-layer search for the container loading problem. *Comput. Oper. Res.* 39 (10), 2267–2276.
- Zhao, X., Bennell, J.A., Bektaş, T., Dowsland, K., 2016. A comparative review of 3D container loading algorithms. *Int. Trans. Oper. Res.* 23 (1–2), 287–320.
- Zhu, W., Huang, W., Lim, A., 2012. A prototype column generation strategy for the multiple container loading problem. *European J. Oper. Res.* 223 (1), 27–39.